

Learn How LLMs Work in 6 Steps

— Then Build One —

THE 6-STEP PIPELINE



Chandra Sekhar
AI Engineer & Trainer

1	Tokenization	BPE, cl100k_base
2	Embeddings	Dense vectors, RoPE
3	Self-Attention	Q·K·V, multi-head
4	Decoding	Temperature, top-p
5	Training	Next-token prediction
6	Production	API & local inference

STEP 1

Tokenization

Raw text is converted into **subword tokens** — the atomic units an LLM understands. Most models use Byte Pair Encoding (BPE).

```
● ● ● tiktoken

import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
text = "Transformers changed NLP forever"
tokens = enc.encode(text)

# [38, 55, 25624, 5614, 483, 452, 18192, 4223]
print(len(tokens)) # 8 tokens
```

- Tokens are subword units, not always whole words
- 1 token $\approx$ 4 characters of English text
- GPT-4 uses cl100k_base: 100,277 token vocabulary
- Token IDs index into the embedding matrix



Check link in caption for AI Handbooks / Projects / Interview Bundle

Context Windows

Context Window = Token Limit. Every token is processed by every attention layer on every forward pass.

GPT-4o ~100k words	128,000 TOKENS
Claude 3.5 Sonnet ~150k words	200,000 TOKENS
Llama 3.1 405B ~100k words	128,000 TOKENS
Gemini 1.5 Pro ~750k words	1,000,000 TOKENS



STEP 2

Embeddings

Each token ID is mapped to a **dense vector** via a learned embedding matrix: `row[token_id]` from `[vocab_size, d_model]`.

 HuggingFace

```
from transformers import AutoTokenizer, AutoModel
model_id = "meta-llama/Llama-3.2-1B"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModel.from_pretrained(model_id)

inputs = tokenizer("Hello world", return_tensors="pt")
embeds = model.embed_tokens(inputs["input_ids"])

print(embeds.shape)
# torch.Size([1, 2, 2048])
#   ^ batch ^ seq_len ^ d_model
```

GPT-2 small

768

D_MODEL

GPT-3 (175B)

12,288

D_MODEL

Llama 3 8B

4,096

D_MODEL

Llama 3 70B

8,192

D_MODEL

 Check link in caption for AI Handbooks / Projects / Interview Bundle

STEP 3

Self-Attention

Each layer: **Multi-Head Attention** + **FFN**, wrapped with residual connections and LayerNorm.

```
● ● ● attention.py

# Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) * V

def attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1))
    scores = scores / (d_k ** 0.5)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V)
```

- Q, K, V are linear projections of the same input
- Multi-Head runs in parallel across h heads, then concatenates
- Causal mask prevents attending to future tokens
- FFN expands to $4 \times d_{\text{model}}$, then projects back
- Llama uses SwiGLU activation instead of ReLU



Check link in caption for AI Handbooks / Projects / Interview Bundle

Transformer Components

Typical sizes for a 7B parameter model:

COMPONENT	ROLE	SIZE
Embedding Matrix	Token ID → vector	32000 × 4096
Q / K / V projections	Input → Q, K, V	4096 × 4096 each
Attention Output	Concat → d_model	4096 × 4096
FFN Up/Gate	Expand features	4096 × 14336
FFN Down	Project back	14336 × 4096
Transformer Layers	Depth of model	32 layers
Attention Heads	Parallel attention	32 heads
KV Heads (GQA)	Grouped queries	8 KV heads



STEP 4

Token Prediction

The LM head maps `d_model` $\rightarrow$ `vocab_size`. Softmax converts logits to probabilities.

 `generate.py`

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

inputs = tokenizer("The transformer architecture",
                    return_tensors="pt")

# Greedy decoding
greedy = model.generate(**inputs, max_new_tokens=20)

# Sampling with temperature + top-p
sampled = model.generate(
    **inputs,
    max_new_tokens=20,
    do_sample=True,
    temperature=0.8,
    top_p=0.95,
)
```



Check link in caption for AI Handbooks / Projects / Interview Bundle

Decoding Methods

Greedy

DETERMINISTIC TASKS

Always pick $\text{argmax}(\text{logits})$

Temperature

CREATIVE GENERATION

Divide logits by T before softmax

Top-k

BALANCED CREATIVITY / COHERENCE

Sample from k highest-prob tokens

Top-p (Nucleus)

MOST POPULAR FOR CHAT MODELS

Sample from tokens summing to $\geq p$

Beam Search

TRANSLATION

Keep top- B candidate sequences



STEP 5

Pretraining

Objective: **next-token prediction** (cross-entropy loss over the vocabulary).

```
● ● ● train.py

# The entire pretraining loss – literally all it is

logits = model(input_ids).logits
targets = input_ids[:, 1:] # next token
logits = logits[:, :-1, :].reshape(-1, vocab_size)

loss = nn.CrossEntropyLoss()(logits, targets.reshape(-1))
loss.backward() # compute gradients
optimizer.step() # update weights
```

- Data: trillions of tokens (CommonCrawl, books, code, Wikipedia)
- Optimizer: AdamW + cosine LR schedule + gradient clipping
- Hardware: thousands of A100/H100 GPUs for months
- Llama 3 405B: 15.6T tokens across 16,000 H100 GPUs



Check link in caption for AI Handbooks / Projects / Interview Bundle

Fine-Tuning

Pretraining creates a base model. Fine-tuning makes it **useful**.

1

SFT (Supervised Fine-Tuning)

Train on (prompt, ideal response) pairs written by humans

2

RLHF

Human raters compare outputs → reward model → optimize via PPO with KL penalty

3

DPO (Direct Preference Optimization)

Newer, simpler: directly train on (chosen, rejected) pairs — no separate reward model

4

LoRA / QLoRA

Parameter-efficient: freeze base model, only train small adapter matrices on Q/K/V/O projections

Try it free: Unsloth + QLoRA on a Colab T4 GPU. Fine-tune Llama 3 in <1 hour.



STEP 6

Using LLMs in Code

OPENAI API (GPT-4O)

```
from openai import OpenAI

client = OpenAI() # reads OPENAI_API_KEY
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user",
                "content": "Explain self-attention"}],
    temperature=0.7,
    max_tokens=512
)
```

LOCAL WITH OLLAMA (NO API KEY)

```
# ollama pull llama3.2

import ollama

response = ollama.chat(
    model="llama3.2",
    messages=[{"role": "user",
                "content": "What is backprop?"}]
)
print(response["message"]["content"])
```



ENJOYED THIS GUIDE?

Share it with your team

One share could save a colleague hours of figuring out how LLMs actually work under the hood.

Repost

Save

Comment

Follow for more AI deep dives